Maquetario

Informe técnico de la rama `version-juan`

Migración del programa de escritorio a Electron, sistema de diseño, seguridad y preparación para la distribución desde Motiova. Qué se hizo, por qué, y qué queda pendiente.

Proyecto	Maquetario 3.24 · creado por Sebastián Poberezny
Preparado por	Juan Cruz Canteros · Tiny Studio
Fecha	7 de noviembre de 2026
Repositorio	pobereznysebastiance/maquetario_2026 · rama version-juan (desde main, commit 479e5f0)
Estado	13 commits · autoprueba completa en verde (23 pasos) · pruebas unitarias y control de diseño en verde

1. Resumen

El objetivo de esta etapa fue llevar Maquetario de una aplicación de Windows funcional a una base que se pueda vender, mantener y hacer crecer: que funcione también en macOS, que se instale desde Motiova con una licencia, que su interfaz tenga un sistema de diseño en lugar de estilos acumulados, y que cada cambio se pueda verificar automáticamente.

No se tocó la lógica editorial: el modelo del libro, la composición, la separación silábica, las reglas editoriales y la salida a PDF siguen siendo las de la versión 3.24. Lo que cambió es el programa que las aloja, la forma en que se organizan los estilos y la infraestructura de calidad alrededor.

Área	Antes	Ahora
Programa de escritorio	C# + WebView2, sólo Windows	Electron + TypeScript; Windows, macOS y Linux. El C# sigue compilando durante la transición.
Distribución	Ejecutable suelto	Un archivo por versión, una carpeta, sin administrador; ejecutable endurecido y con verificación de integridad.
Licencias	No existían	Formato de licencia firmada (Ed25519) verificable sin conexión, listo para el launcher de Motiova.
Estilos	23 hojas que se pisaban entre sí	Sistema de diseño por capas (tokens, átomos, moléculas, organismos, plantillas) con control automático.
Verificación	Autoprueba en C# con ventanas visibles	Autoprueba de 23 pasos sobre la app real (PDF incluidos), capturas en 5 resoluciones y pruebas unitarias.

2. Programa de escritorio

2.1 Electron en lugar de C#/WebView2

Decisión	Reemplazar el host en C# por Electron con el proceso principal en TypeScript. La interfaz (HTML, CSS y JS de `LibroStudio/v3/app`) se mantiene tal cual.
Motivo	La maquetación se compone con el motor del navegador. Para que un libro corte las líneas igual en todas las computadoras hace falta el mismo motor en todas: Electron lleva el mismo Chromium a Windows y macOS. Además resuelve instaladores, firma, notarización y actualizaciones con herramientas maduras.
Alternativas	**Tauri**: binario más chico, pero en macOS usa WebKit y las líneas pueden cortar distinto; descartado para una herramienta de maquetación. **Seguir en C#**: limita a Windows, y buena parte de los diseñadores editoriales trabaja en Mac.
Consecuencias	El programa pesa más (unos 320 MB sin comprimir). A cambio, la interfaz no se reescribió: se mantuvo el mismo protocolo de mensajes que usaba WebView2.

2.2 Mismo protocolo, una capa de host

Decisión	La interfaz habla con el programa sólo a través de `host.js` (`MaquetarioHost`), que detecta si corre en Electron, en el C# anterior o en un navegador. Las 26 acciones del C# (guardar, abrir, fuentes, PDF, impresión, paquete, asistente, portapapeles) se portaron una por una a módulos de `desktop/src/main/handlers`.
Motivo	Permite migrar sin cortar el desarrollo: la misma interfaz funciona en los dos programas mientras dure la transición, y el resto del código no sabe en qué programa corre.
Consecuencias	El lector de fuentes OpenType se portó de C# a TypeScript conservando el mismo identificador de cada fuente, para que los proyectos y paquetes existentes sigan encontrando sus tipografías.

2.3 Datos y perfiles

Decisión	Preferencias, recientes y carpetas de fuentes en la carpeta de datos del usuario; la autoprueba usa un perfil propio.
Motivo	Al compartir perfil, la autoprueba heredaba preferencias de la persona (por ejemplo, el modo continuo) y chocaba con una ventana abierta. Es el mismo criterio que ya tenía la versión C# con `BrowserData-selftest`.

---

3. Seguridad y distribución

El criterio fue que la página no pueda hacer nada que no esté expresamente permitido, y que el programa instalado no se pueda modificar ni usar para otra cosa.

Medida	Qué evita
Aislamiento de contexto, sandbox global (`app.enableSandbox`), sin integración con Node	Que código de la página acceda al sistema.
Esquema propio `maquetario://` con control de rutas	Que se lean archivos fuera de la carpeta de la interfaz.
Lista cerrada de 26 acciones y validación del remitente de cada mensaje	Que la página pida operaciones no previstas.
Política de contenido (meta y cabecera), sin red, sin plugins ni marcos, `nosniff`	Ejecución de código inyectado y conexiones externas.
Navegación, redirecciones, ventanas nuevas y permisos bloqueados	Que la app salga de su propio contenido.
Fuses de Electron: sin modo Node, sin `NODE_OPTIONS`, carga sólo desde `app.asar` con verificación de integridad	Usar el ejecutable para correr otro código o modificar la interfaz instalada. Verificado: con un byte cambiado en el paquete, la app no abre.
Límites de tamaño en guardar y exportar	Bloqueos por archivos desmedidos.

3.1 Licencias firmadas

Decisión	Formato `MQL1.<datos>.<firma>` con firma Ed25519 (`packages/license`). Motiova firma con su clave privada; Maquetario verifica con la pública, sin conexión, hasta el vencimiento. Incluye producto, edición, versión mayor cubierta, equipo y vencimiento.
Motivo	La clave privada nunca sale del servidor, así que la licencia no se puede fabricar. La verificación local permite trabajar sin internet durante el período de la licencia.
Estado	Implementado y probado (licencia válida, vencida, de otra versión y faltante). Queda en modo apagado hasta que exista el launcher; el modo se fija al compilar y no se puede cambiar en una instalación.

3.2 Instalación desde Motiova

Decisión	El launcher descarga un único archivo por versión, verifica su firma y lo descomprime en una carpeta (`Motiova/apps/maquetario/<versión>`). Las versiones se instalan una al lado de la otra para poder volver atrás. Al iniciar sesión, el launcher consulta si hay una versión nueva. También se genera un instalador de un clic, por usuario y sin pedir administrador.
Motivo	Instalación simple, sin pasos adicionales (Electron no necesita WebView2 ni .NET) y actualizaciones controladas por quien tiene la sesión.
Pendiente	Certificado de firma de código. Sin firma, Windows muestra "editor desconocido"; pedir permisos de administrador no lo evita. Opciones: Azure Trusted Signing o un certificado OV/EV en Windows, y cuenta de Apple Developer con notarización en macOS.

4. Sistema de diseño

El diagnóstico inicial fue que la interfaz no tenía un sistema, sino una acumulación: cada función nueva traía su hoja de estilos y pisaba a la anterior (`studio.css` , luego `maquetario.css` , `glass.css` , `crystal.css`).

Medida	Diagnóstico inicial	Hoy
Colores distintos escritos a mano	633	0 en el sistema nuevo
Archivos que redefinían el botón	15	1 componente
Tamaños de letra distintos	34 (88 usos de 7 a 10 px)	escala de 9 roles
Radios distintos	31	3 (4, 6 y 8 px)
Cadenas de `onchange` del idioma pisándose	7	0

4.1 Capas de cascada y atomic design

Decisión — Un único punto de entrada (`ui/index.css`) con capas: `legacy` < tokens < base < átomos < moléculas < organismos < plantillas. Todo el CSS anterior quedó en la capa `legacy` , sin borrar nada.

Motivo — Lo nuevo gana sin `!important` y sin pelear especificidad, y se puede migrar pantalla por pantalla sin romper el resto. Cuando una pieza ya no depende de `legacy` , su regla vieja se borra.

Componentes — Átomos: botón, campo, ícono, esqueleto de carga. Moléculas: acción con descripción, fila de preferencia, opción de menú, casilla, avisos con estado, manija y plegado de secciones. Organismos: barra superior, paneles, inspector, diálogo, menú, barra de estado, tira de páginas, bienvenida, asistente, entre otros. El marcado de los componentes se genera desde `ui/components.js` , con la misma API que tendrán como componentes de React.

4.2 Tokens y control automático

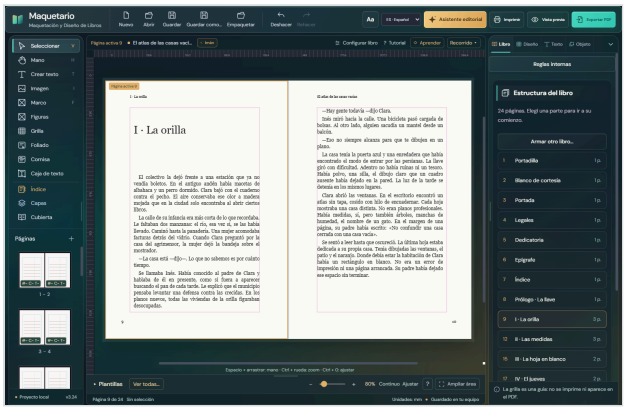
Decisión — Colores, espacios, tipografía, radios, sombras y materiales viven en `ui/tokens` . `npm run lint:design` falla si aparece un color o una medida suelta en el sistema nuevo, e informa la deuda que queda en `legacy` .

Motivo — Sin un control, el sistema se vuelve a degradar con el primer apuro. Con él, la deuda sólo puede bajar.

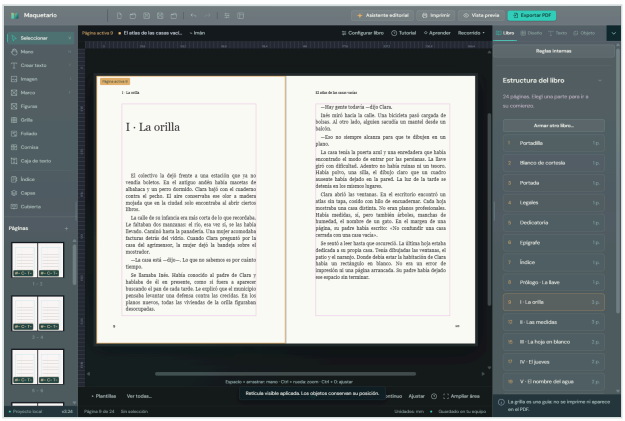
4.3 Identidad y criterio visual

Decisión	Paleta del instalador (petróleo, verde agua, filete dorado, DM Sans); tipografía con los roles de la escala de macOS (Footnote, Callout, Body, Headline, Title 3/2/1); radios sutiles; material translúcido real del sistema (Mica en Windows 11, vibrancy en macOS) en lugar de vidrio dibujado con CSS; colores de estado derivados de la marca (ladrillo, dorado, salvia, verde agua claro).
Motivo	El vidrio dibujado con desenfoques apilados era caro de pintar y genérico. El material del sistema lo calcula el sistema operativo. La página del libro nunca queda detrás de un vidrio, para juzgar bien el color.
Respetado	Las pruebas de legibilidad existentes (títulos de sección de 16 px o más, ayudas de 12 px o más) siguen pasando; la escala se ajustó a ellas.

Antes

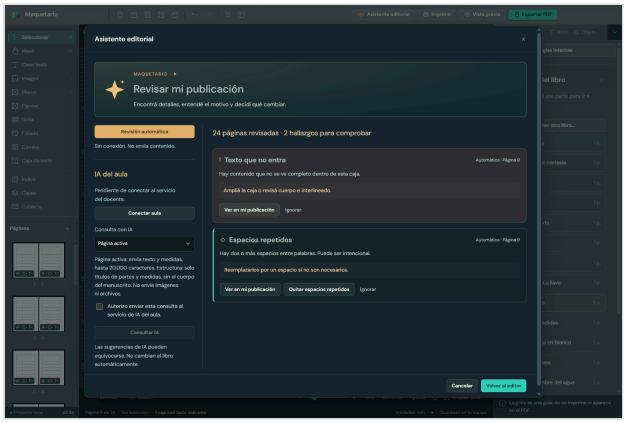


Ahora

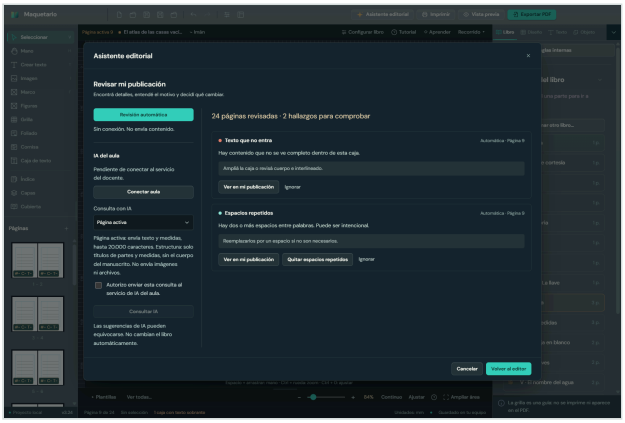


Editor con el libro de ejemplo. Barra compacta, paneles pegados al borde, inspector por secciones sin tarjetas anidadas. La página del libro no cambia.

Antes



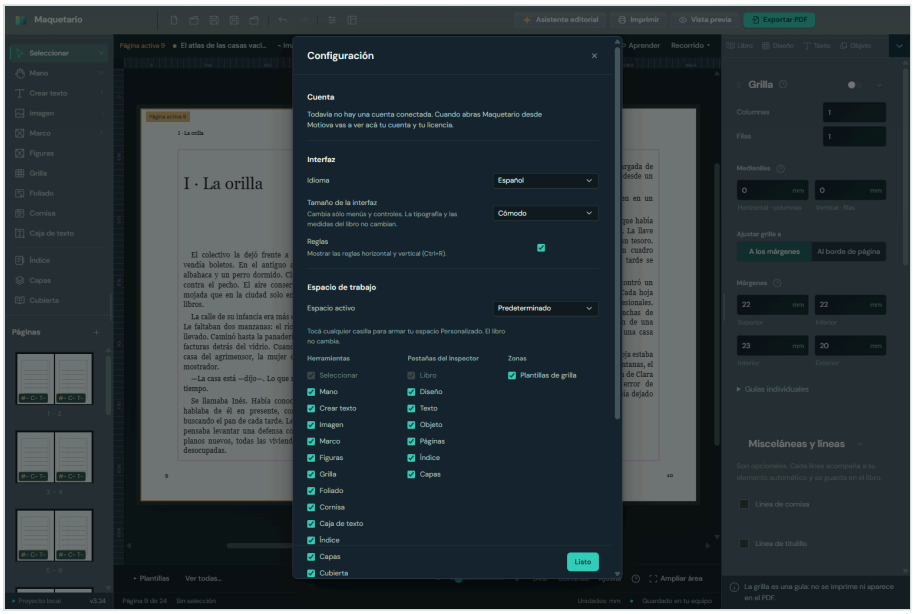
Ahora



Asistente editorial. Se quitaron la tarjeta con degradé, la etiqueta superior y el dorado forzado; los hallazgos se leen como filas con un punto de estado.

5. Funciones nuevas en la interfaz

- **Configuración**: cuenta y licencia de Motiova, idioma, tamaño de la interfaz, reglas, espacio de trabajo, carpetas de fuentes y datos de la versión. Reúne preferencias que estaban sueltas en la barra.
- **Espacios de trabajo**: Inicial (lo esencial para empezar), Predeterminado (todo) y Personalizado (selección guardada). Pensado para que un estudiante no se encuentre con todas las herramientas el primer día.
- **Inspector**: secciones que se reordenan arrastrando o con Alt + flechas, y que se pliegan; el estado se recuerda por pestaña.
- **Confirmaciones** para acciones que pierden trabajo, con el foco en Cancelar; mientras están abiertas, los atajos de la app no reciben teclas.
- **Reglas ocultables** con Ctrl+R, como en InDesign.
- **Barra de título integrada** a la barra de Maquetario, respetando el área real de los botones de ventana en cualquier escala de pantalla.



Configuración: secciones de arriba hacia abajo, una fila por preferencia.

6. Arquitectura interna de la interfaz

6.1 Pruebas fuera del programa distribuido

Decisión	Las 13 hojas de pruebas ya no se cargan en `index.html`; las carga `test-loader.js` sólo en modo autoprueba, y el paquete distribuido no las incluye.
Motivo	El código de prueba no debe viajar al usuario. Se verificó antes que ningún módulo de producción dependiera de ellas.

6.2 Del reemplazo de funciones globales a eventos

Decisión	Las 7 cadenas en las que cada módulo pisaba el `onchange` del selector de idioma pasaron a escuchar un evento (`maquetario:language`). El orden de ejecución es el mismo que antes.
Motivo	Con el patrón anterior, el comportamiento dependía del orden de 65 etiquetas `<script>` y un módulo podía romper a otro sin aviso. Es el primer caso resuelto de un patrón que todavía aparece 127 veces (ver sección 8).

7. Calidad y verificación

Herramienta	Qué verifica
`npm run self-test`	23 pasos sobre la app real: pruebas de integración de la interfaz, guardado, recientes, fuentes del sistema, siete PDF reales (lectura, índice, marcas, tipografía, párrafos, novela, cubierta), confirmaciones, espacios de trabajo, reglas, paquete del proyecto y capturas en 1366×768, 1536×864, 1920×1080, 2560×1440 y 3840×2160 con control de desborde horizontal. Cada paso tiene límite de tiempo.
`npm test`	Licencias (8 casos), lector de fuentes OpenType (4 casos) y control del sistema de diseño.
`npm run typecheck`	Tipos del programa de escritorio y del paquete de licencias.

La autoprueba también corre sobre el ejecutable empaquetado. Se comparó la salida PDF con la de la versión C#: mismas páginas y formatos. El tamaño de hoja puede diferir entre 0,1 y 0,3 mm en ambas versiones por redondeo de Chromium; se corrige fijando las cajas del PDF en la etapa de salida para imprenta.

8. Pendientes y decisiones abiertas

Deuda medida (sale en cada `npm test`)

Deuda en el código heredado	Cantidad
Funciones globales reemplazadas por otro módulo	127
Anchos fijos en px	137
Estilos escritos desde JavaScript	111
`!important`	63
Colores escritos a mano en la capa legacy	965

Etapas siguientes

1. Terminar la migración de pantallas al sistema de diseño (cubierta, tutorial, glosario, índice, capas) y selectores propios.

2. TypeScript y módulos ES para la interfaz, eliminando los reemplazos de funciones globales; migraciones explícitas del formato `.mqt`.
3. Componentes de React sobre los mismos tokens y clases, con revisión de experiencia de uso (observaciones en `docs/evaluacion-uso.md`).
4. Motor de composición propio (HarfBuzz, justificación Knuth-Plass) y PDF para imprenta: CMYK, PDF/X, cajas de corte y sangrado exactas.
5. Launcher y backend en Motiova: sesión, accesos a recursos, descargas firmadas y emisión de licencias.

Decisiones que necesitan definición

- Qué controla la licencia: edición o plan, o número de versión comprada. El formato admite ambas.
- Pago único o suscripción, y cuántos días puede funcionar sin conexión.
- Compra del certificado de firma de código y de la cuenta de Apple Developer.

---

9. Cómo probarlo

En la raíz del repositorio, con Node 22 o posterior:

`npm install`	Una vez.
`npm run dev`	Abre Maquetario en modo desarrollo. Los cambios de interfaz se ven con Ctrl+R; las herramientas de desarrollo con F12.
`npm test` · `npm run self-test`	Pruebas unitarias y autoprueba completa.
`npm run dist`	Instalador y archivo para el launcher en `desktop/release`.

La documentación de referencia está en `docs/arquitectura.md` (estructura, contrato con el launcher, instalación y firma), `docs/plan-interfaz.md` (sistema de diseño) y `PRODUCT.md` (usuarios, propósito y criterios del producto).